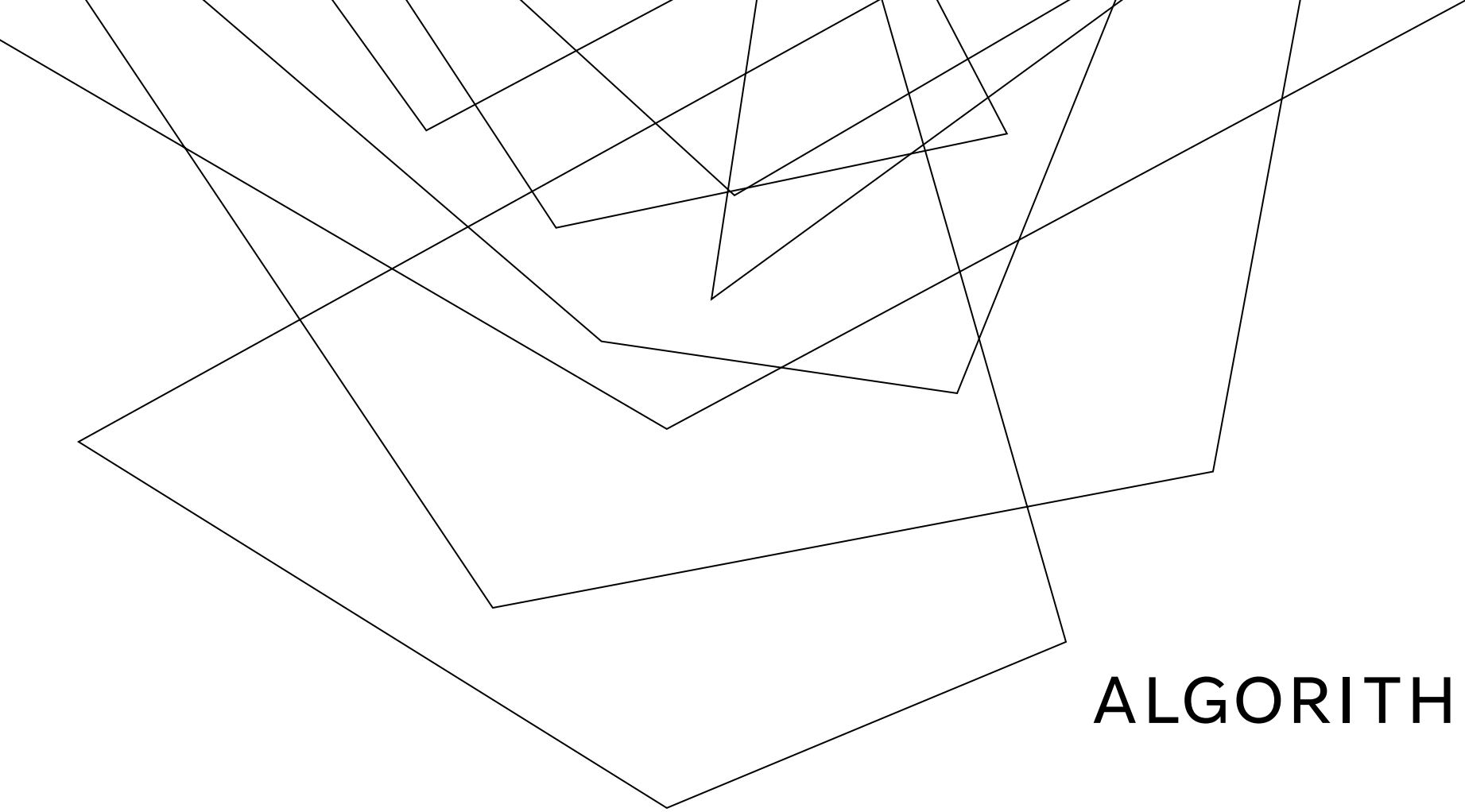


A series of thin, black, overlapping lines forming various geometric shapes like triangles and polygons, creating a complex, abstract pattern in the upper left portion of the slide.

ALGORITHMS

9. DYNAMIC PROGRAMMING (1)





지난 시간 복습

SELECTION

1. Selection problem
2. Quickselect
3. Median of medians



QUICKSELECT

이론

$$T(n) \leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} T(k) + c_0 n$$

$$\leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} ck + c_0 n \quad (\text{귀납 가설})$$

$$= \frac{c}{n} \left(n - 1 + \left\lfloor \frac{n}{2} \right\rfloor \right) \left(n - \left\lfloor \frac{n}{2} \right\rfloor \right) + c_0 n$$

$$\sum_{k=m}^{n-1} k = \frac{(n-1+m)(n-m)}{2}$$



MEDIAN OF MEDIANS (+ QUICKSELECT)

Complexity analysis

- Worst case : $O(n)$

Guess & Verification 방법으로 증명 가능

$$\begin{aligned}T(n) &\leq T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + c_0n \\&\leq c\frac{n}{5} + c\frac{7n}{10} + c_0n && \text{귀납 가설} \\&= c\frac{9}{10}n + c_0n \\&= cn - \left(\frac{c}{10} - c_0\right)n \leq cn && \text{조건: } 10c_0 \leq c\end{aligned}$$





목차

목차

1. Dynamic Programming
2. Matrix Path Problem
3. Matrix Multiplication Order Problem



An abstract graphic design featuring two thin, dark gray lines that intersect on a light gray background. One line is oriented diagonally from the top-left towards the bottom-right, while the other is oriented from the top-right towards the bottom-left. The intersection point is located to the left of the text.

DYNAMIC PROGRAMMING

DYNAMIC PROGRAMMING

- 큰 문제의 해답에 작은 문제의 해답이 포함되어 있고
(= 최적 부분 구조, Optimal Substructure)
- 이를 재귀 호출 알고리즘으로 구현하면 지나친 중복 호출이 발생하는 경우
- 적절한 저장 방법으로 중복 호출의 비효율을 제거하는 알고리즘이 DP



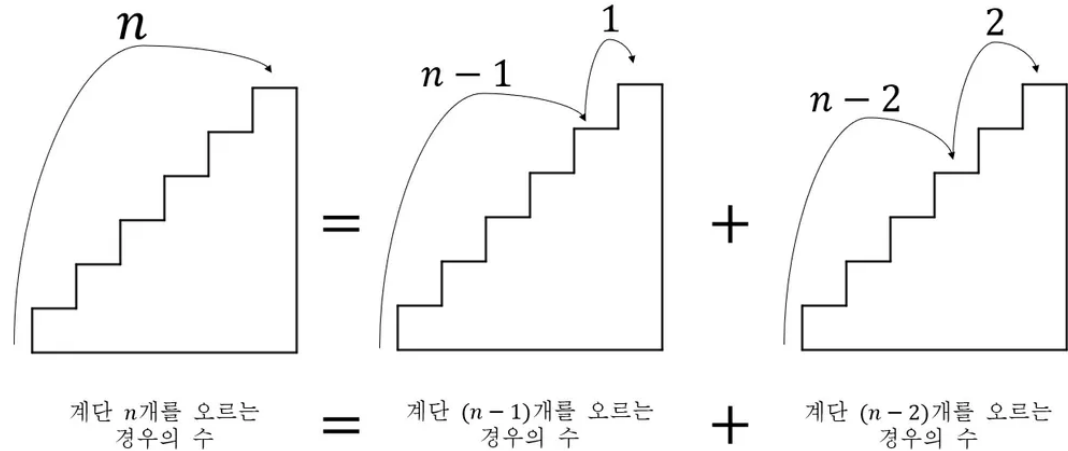
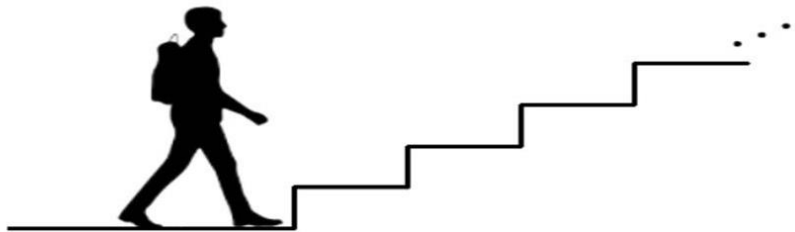
DYNAMIC PROGRAMMING

Ex) Fibonacci numbers

- 피보나치 수: 앞의 두 숫자를 더해서 다음 숫자를 만드는 수열
 - $\text{fib}(0) = 0, \text{fib}(1) = 1$
 - 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

계단오르는 방법 (경우의 수)

한 계단 혹은 두 계단씩 오를 때,
10개의 계단을 오르는 경우는 모두
몇 가지인가?



DYNAMIC PROGRAMMING

Q. n번째 피보나치 수를 구하는 다음 함수의 시간복잡도는?

```
def fib(n):  
    if n <= 1: return n  
    return fib(n-1) + fib(n-2)
```

→ 큰 문제의 해답에 작은 문제의 해답이 포함되어 있음



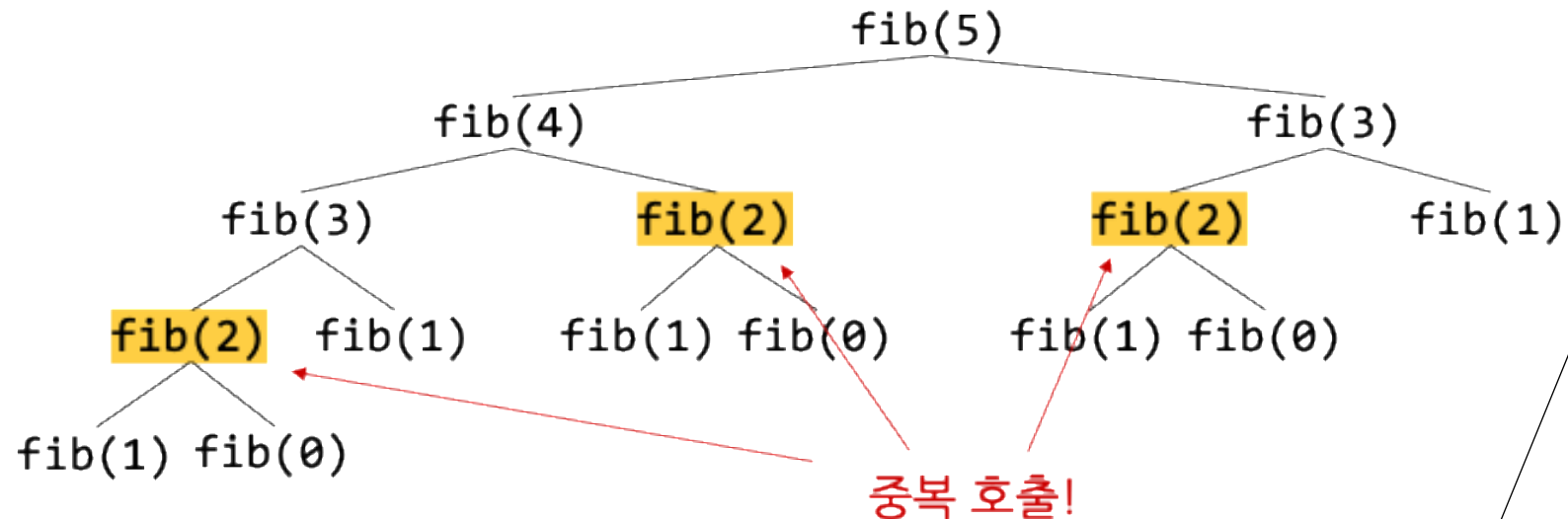
DYNAMIC PROGRAMMING

Q. n번째 피보나치 수를 구하는 다음 함수의 시간복잡도는?

```
def fib(n):  
    if n <= 1: return n  
    return fib(n-1) + fib(n-2)
```

A. $\Theta(\phi^n)$, $\phi = 1.618 \dots$

Why? 중복 호출이 지나치게 발생해서



DYNAMIC PROGRAMMING

Q. 어떻게 중복 호출을 줄이고 효율적으로 함수를 개선할까?

A. 작은 문제의 해답을 테이블에 “저장”해두고 “재사용”

- Bottom-up 방식 (Tabulation) : for문 loop로 구현
작은 문제부터 차근차근 풀기

Bottom-up 방식 (Tabulation)

```
def fib(n):  
    memo = [0, 1] + [-1] * n  
    for i in range(2, n+1):  
        memo[i] = memo[i-1] + memo[i-2]  
    return memo[n]
```



DYNAMIC PROGRAMMING

Q. 어떻게 중복 호출을 줄이고 효율적으로 함수를 개선할까?

A. 작은 문제의 해답을 테이블에 “저장”해두고 “재사용”

- Top-down 방식 (Memoization) : 재귀 방식으로 구현

최초 호출되면 테이블에 해답을 “메모” 해둠

이후에 호출되면 저장된 테이블 값을 return

Top-down 방식 (Memoization)

```
memo = [0, 1] + [-1] * n
def fib(n):
    if memo[n] != -1: return memo[n]
    memo[n] = fib(n-1) + fib(n-2)
    return memo[n]
```



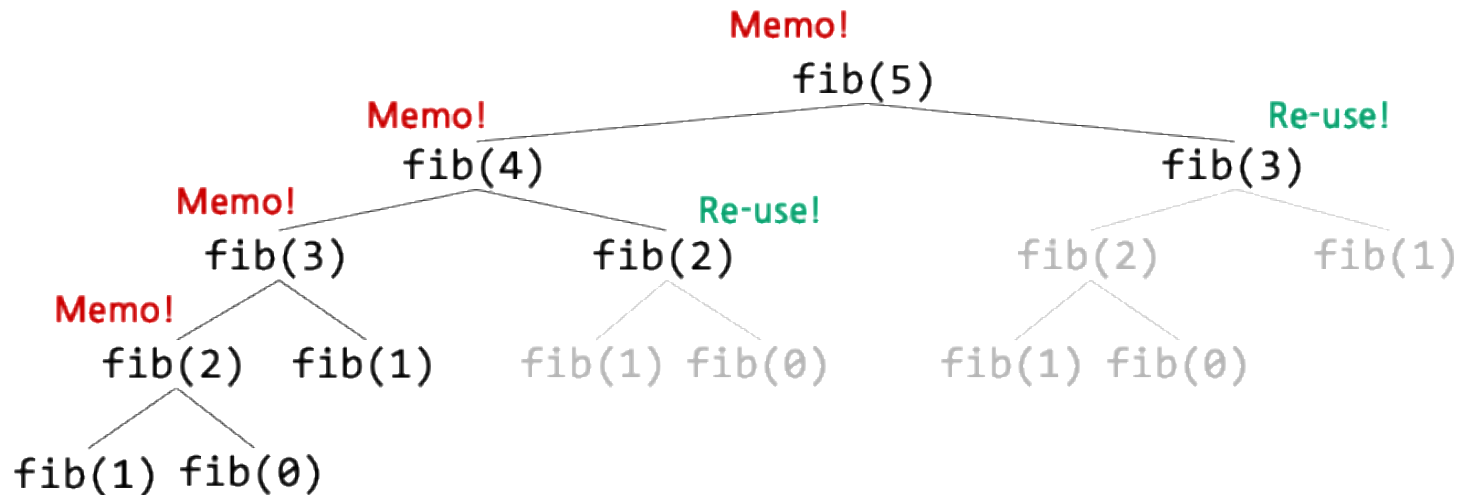
DYNAMIC PROGRAMMING

Q. 어떻게 중복 호출을 줄이고 효율적으로 함수를 개선할까?

A. 작은 문제의 해답을 테이블에 “저장”해두고 “재사용”

- Top-down 방식 (Memoization) : 재귀 방식으로 구현

최초 호출되면 테이블에 해답을 “메모” 해둠
이후에 호출되면 저장된 테이블 값을 return



MATRIX PATH
PROBLEM

MATRIX PATH PROBLEM

$n \times n$ 행렬이 주어진다. 행렬의 왼쪽 위에서 시작해 한 칸씩 이동해 오른쪽 아래까지 도달한다. 이 과정에서 방문한 칸에 있는 수들을 더한 값이 이 경로의 합이다. 이동 규칙은 다음과 같다.

- 오른쪽이나 아래쪽으로만 이동할 수 있다.
- 왼쪽, 위쪽, 대각선 이동은 허용하지 않는다.

왼쪽 위 끝인 $(0, 0)$ 에서 오른쪽 아래 끝인 $(n-1, n-1)$ 로 이동하는 모든 경로의 점수 중 가장 높은 점수를 구하시오.

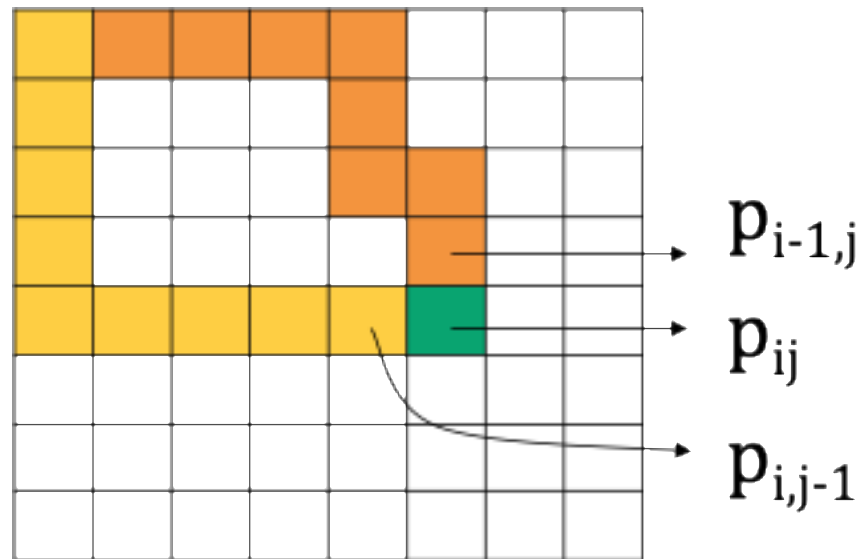
6	7	12	5
5	3	11	18
7	17	3	3
8	10	14	9



MATRIX PATH PROBLEM

- 재귀 특성 파악 (점화식 세우기)

$$p_{ij} = 0 \text{ if } i < 0 \text{ or } j < 0$$
$$p_{ij} = \max(p_{i,j-1}, p_{i-1,j}) + m_{ij}$$



MATRIX PATH PROBLEM

- Bottom-up 방식 (Tabulation) 코드

```
def matrix_path(mat):  
    n = len(mat)  
    dp = [[0] * n+1 for _ in range(n+1)]  
  
    for i in range(n):  
        for j in range(n):  
            dp[i][j] = max(dp[i][j-1], dp[i-1][j]) + mat[i][j]
```

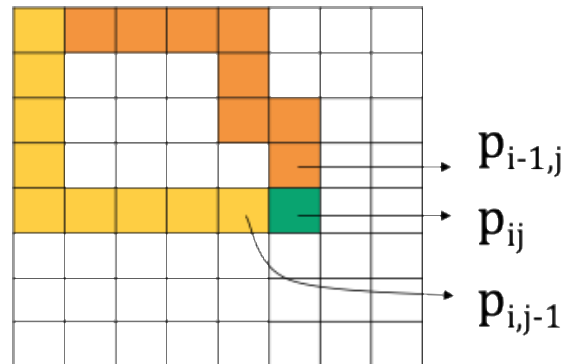
6	7	12	5
5	3	11	18
7	17	3	3
8	10	14	9



MATRIX PATH PROBLEM

- Top-down 방식 (Memoization) 코드

```
def matrix_path(mat):  
    n = len(mat)  
    dp = [[0] * n for _ in range(n)]  
  
    def rec(i,j):  
        if i < 0 or j < 0: return 0  
        if dp[i][j] != 0: return dp[i][j]  
        dp[i][j] = max(rec(i-1,j), rec(i,j-1)) + mat[i][j]  
        return dp[i][j]  
  
    return rec(n-1,n-1)
```





MATRIX MULTIPLICATION ORDER PROBLEM

MATRIX MULTIPLICATION ORDER PROBLEM

n 개의 행렬 $A_0, A_1, \dots, A_{n-1}$ 이 주어지고, 이들의 곱 $A_0A_1\dots A_{n-1}$ 을 구하려고 한다. 연속한 두 행렬을 반복하여 곱해나가는 방식으로 궁극적으로 하나의 행렬 $A_0A_1\dots A_{n-1}$ 을 구할 수 있다. 행렬을 곱하는 순서에 따라 스칼라곱 연산량이 다른데, 행렬 $A_0A_1\dots A_{n-1}$ 를 구하는데 필요한 가장 적은 스칼라곱 연산량을 구하시오.

단, 연속한 두 행렬 A_i 와 A_{i+1} 의 크기가 각각 $p_i \times p_{i+1}$, $p_{i+1} \times p_{i+2}$ 라면 이 두 행렬을 곱할 때 $p_i p_{i+1} p_{i+2}$ 의 스칼라곱 연산량을 필요로 한다.

cf) 행렬곱 연산은 결합법칙이 성립, 즉 $A_0A_1A_2 = (A_0A_1)A_2 = A_0(A_1A_2)$

- 입력: $p_0, p_1, \dots, p_n$ (A_i 는 $p_i \times p_{i+1}$ 행렬)

$$\begin{matrix} \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} & \times & \begin{bmatrix} 6 & 3 \\ 5 & 2 \\ 4 & 1 \end{bmatrix} & = & \begin{bmatrix} 1 \times 6 + 2 \times 5 + 3 \times 4 & 1 \times 3 + 2 \times 2 + 3 \times 1 \\ 4 \times 6 + 5 \times 5 + 6 \times 4 & 4 \times 3 + 5 \times 2 + 6 \times 1 \end{bmatrix} \\ 2 \times 3 & & 3 \times 2 & & 2 \times 3 \times 2 = 12 \text{ scalar multiplications} \end{matrix}$$



MATRIX MULTIPLICATION ORDER PROBLEM

Ex) $A_0(10 \times 100)$, $A_1(100 \times 5)$, $A_2(5 \times 50)$ 인 경우

- **Case 1: $(A_0 \times A_1) \times A_2 \Rightarrow 7,500$ 스칼라 곱 연산 수행**
 - $A_0 \times A_1(=A_{01}): 10 \times 100 \times 5 = 5000$
 - $A_{01} \times A_2: 10 \times 5 \times 50 = 2500$
- **Case 2: $A_0 \times (A_1 \times A_2) \Rightarrow 75,000$ 스칼라 곱 연산 수행**
 - $A_1 \times A_2(=A_{12}): 100 \times 5 \times 50 = 25000$
 - $A_0 \times A_{12}: 10 \times 100 \times 50 = 50000$

➔ 곱셈 순서에 따라 스칼라곱 연산량이 크게 달라짐



MATRIX MULTIPLICATION ORDER PROBLEM

- 재귀 특성 파악 (점화식 세우기)

$S(i, j)$ 을 $A_i A_{i+1} \dots A_{j-1}$ 의 최소 연산량이라 하면
 $S(s, e) = \min(S(s, i) + S(i, e) + p_s p_i p_e \text{ for } s < i < e)$

Ex) $A_1 A_2 A_3 A_4 A_5 A_6 A_7$ 문제의 경우

$\left. \begin{array}{l} (A_1)(A_2 A_3 A_4 A_5 A_6 A_7) \\ (A_1 A_2)(A_3 A_4 A_5 A_6 A_7) \\ (A_1 A_2 A_3)(A_4 A_5 A_6 A_7) \\ (A_1 A_2 A_3 A_4)(A_5 A_6 A_7) \\ (A_1 A_2 A_3 A_4 A_5)(A_6 A_7) \\ (A_1 A_2 A_3 A_4 A_5 A_6)(A_7) \end{array} \right\}$

이들 중 최소 연산량을
보이는 것이 정답!

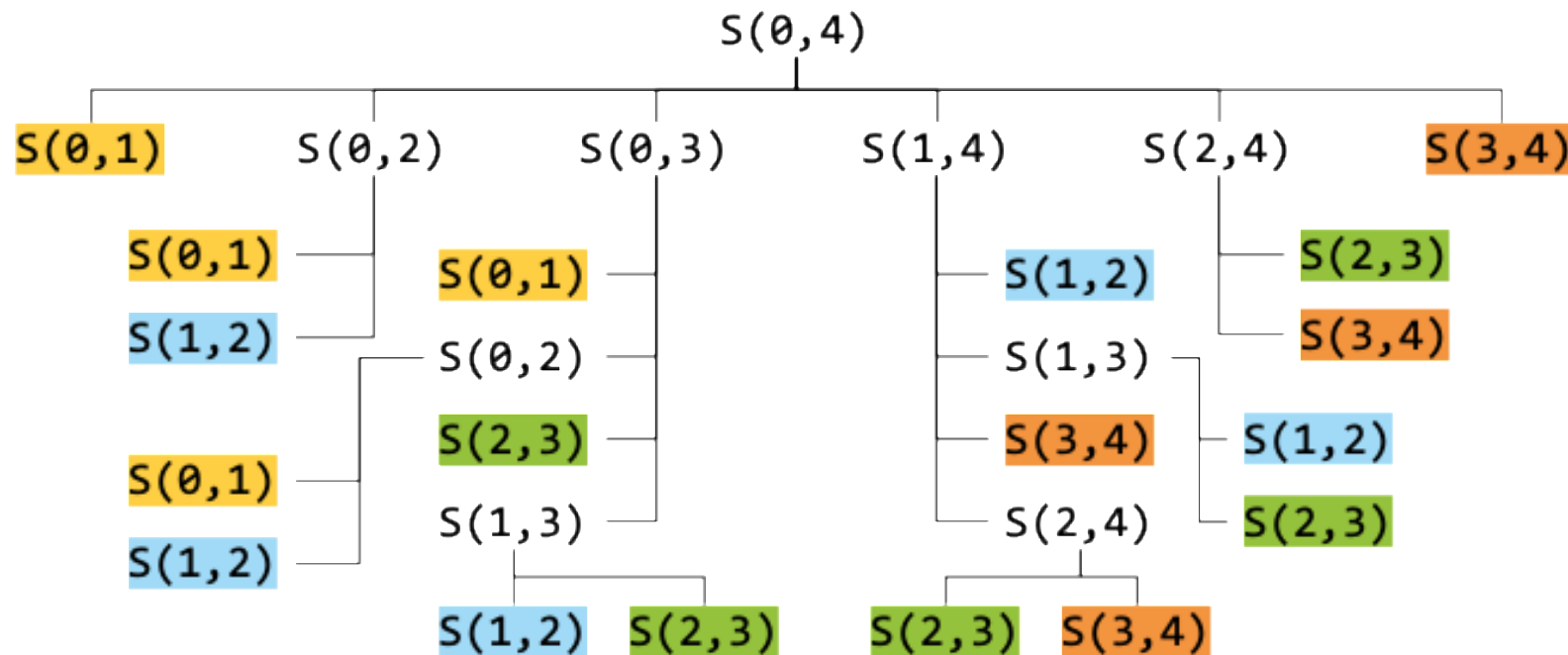


MATRIX MULTIPLICATION ORDER PROBLEM

Ex) $S(0,4)$ 함수의 호출 관계 예시

→ 중복 호출로 심각한 비효율 발생

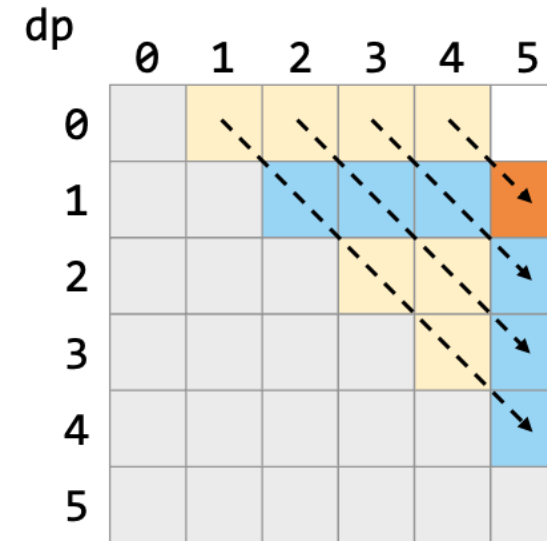
→ Dynamic Programming 필요



MATRIX MULTIPLICATION ORDER PROBLEM

- Bottom-up 방식 (Tabulation) 코드

```
def mmo(p):  
    n = len(p)  
    dp = [[math.inf] * n for _ in range(n)]  
  
    for s in range(n-1):  
        dp[s][s+1] = 0  
  
    for r in range(2,n):  
        for s in range(n-r):  
            e = s+r  
            for i in range(s+1,e):  
                dp[s][e] = min(dp[s][e], dp[s][i] + dp[i][e] + p[s]*p[i]*p[e])  
  
    return dp[0][n-1]
```



MATRIX MULTIPLICATION ORDER PROBLEM

- Top-down 방식 (Memoization) 코드

```
def mmo(p):  
    n = len(p)  
    dp = [[-1] * n for _ in range(n)]  
  
    def S(s, e):  
        if e - s == 1: return 0  
        if dp[s][e] != -1: return dp[s][e]  
  
        ans = math.inf  
        for i in range(s+1, e):  
            ans = min(ans, S(s, i) + S(i, e) + p[s]*p[i]*p[e])  
  
        dp[s][e] = ans  
        return ans  
  
    return S(0, len(p)-1)
```



An abstract graphic featuring two thin, dark lines that intersect on a light gray background. One line is oriented diagonally from the top-left towards the bottom-right, while the other is more horizontal, sloping slightly downwards from left to right. The intersection point is located in the upper-left quadrant of the image. To the right of this intersection, the word "SUMMARY" is written in a bold, black, sans-serif typeface.

SUMMARY

SUMMARY

- 1. Dynamic Programming**
- 2. Matrix Path Problem**
- 3. Matrix Multiplication Order Problem**



NEXT LECTURE

Dynamic programming (2)



Two thin, dark gray lines intersect diagonally on a light gray background. One line runs from the top-left towards the bottom-right, and the other runs from the top-right towards the bottom-left. They cross each other in the upper-left quadrant of the image.

QUESTIONS?